Schema Evolution and Data Contract Enforcement for Reliable ML Pipelines Using NYC TLC Trip Records

Technical Report

Student 1: Deana Naveen A	SRN 1: PES1UG24AM077
Student 2: G Ganikhasri	SRN 2: PES1UG24AM098
Student 3: Janavi Agrawal	SRN 3: PES1UG24AM120

Abstract

Modern data-driven applications depend on data pipelines that remain reliable as datasets evolve over time. Changes in a dataset's schema, such as adding or removing fields or changing data types, can cause downstream processing failures or negatively affect machine learning systems if they are not detected appropriately. This project investigates schema evolution and data contract enforcement using the New York City Taxi and Limousine Commission (NYC TLC) Yellow Taxi Trip Records.

January 2024 and January 2025 NYC TLC datasets were compared to identify structural changes. The January 2024 dataset contained 19 columns, while the January 2025 dataset contained 20 columns. The additional field was cbd_congestion_fee. No columns were removed and no data type changes were detected among shared fields.

A data contract was then created using the January 2024 schema as the baseline. Contract validation was implemented to verify required fields and their data types while allowing compatible additional columns. Automated tests were developed to verify that missing required fields and incompatible data types are rejected while permitted additional fields are accepted.

Finally, a machine learning experiment was conducted using a regression model to predict taxi fare amounts from trip_distance and passenger_count. The baseline model achieved an MAE of 2.9068 and an RMSE of 7.3373. A controlled data-corruption experiment in which trip_distance values were replaced with zero increased MAE to 14.1528 and RMSE to 20.1635. These results demonstrate the potential impact of corrupted input data on downstream ML performance and motivate contract enforcement before model execution.

1. Introduction

Data pipelines frequently process datasets that change over time. New business requirements, regulatory changes, or additional data collection can introduce new columns, remove existing fields, or modify data types. While some changes are backward compatible, others can break downstream applications or silently reduce the quality of machine learning predictions.

A major challenge is distinguishing between compatible schema evolution and breaking schema changes. For example, adding a new optional field may be acceptable for a downstream consumer that only depends on a fixed set of existing fields. In contrast, removing a field required by a machine learning model can prevent the model from operating correctly or cause an implementation to substitute an inappropriate default value.

This project investigates these issues using NYC TLC Yellow Taxi Trip Records. The project focuses on three related objectives: (1) identify schema changes between different versions of a real-world dataset; (2) define and enforce a data contract that distinguishes acceptable and breaking changes; and (3) demonstrate how a data-quality failure involving a required ML feature can affect model performance.

2. Dataset and Problem Definition

2.1 Dataset

The project uses NYC TLC Yellow Taxi Trip Records for January 2024 and January 2025. These records contain information about taxi trips, including trip distance, passenger count, fare amount, and other trip-related attributes.

The datasets were stored in Parquet format and processed using Python and PyArrow. Parquet was used because it provides a structured, column-oriented representation that allows the project to inspect schemas efficiently without loading the complete dataset into memory. The January 2024 dataset was used as the baseline for defining the initial data contract.

2.2 Problem Definition

The central problem addressed by this project is: How can schema changes in evolving datasets be detected and controlled before they negatively affect downstream data processing and machine learning systems?

The project considers three types of schema changes: added columns, removed columns, and data type changes. These changes do not necessarily have the same level of risk. An additional optional column can often be accepted, whereas removing a required model feature should be treated as a breaking change. The system therefore implements explicit rules rather than treating every schema difference as an error.

3. Methodology

The project consists of four major stages: schema comparison; data contract creation and validation; automated contract testing; and machine learning impact evaluation.

3.1 Schema Comparison

The first stage compares the schemas of the January 2024 and January 2025 datasets. The comparison checks the number of columns, columns added in the newer dataset, columns removed from the newer dataset, and data type changes among common columns.

Property	January 2024	January 2025
Number of columns	19	20
Columns added	—	cbd_congestion_fee
Columns removed	—	None
Shared columns with changed types	—	None

The addition of cbd_congestion_fee represents an example of schema evolution. Because the field was added rather than replacing an existing field, the change can be treated as compatible under the project's current contract policy.

3.2 Data Contract

A data contract was created using the January 2024 schema as the baseline. The contract specifies required column names, expected data types, and whether additional columns are permitted.

The project uses the following compatibility policy: required columns and their data types must remain compatible, while additional columns are permitted. Under this policy, the January 2025 dataset remains valid because it contains all required baseline fields and introduces only an additional field.

The data contract therefore separates schema evolution from schema incompatibility. This distinction is important because automatically rejecting every additional field would make a data pipeline unnecessarily fragile.

3.3 Contract Validation

A Python-based validator was implemented to compare an incoming Parquet dataset against the contract. The validator checks whether every required field exists, whether required fields have the expected data type, and whether additional fields are permitted by the contract.

Dataset	Status	Missing Columns	Additional Columns	Type Changes
January 2024	VALID	0	0	0
January 2025	VALID	0	cbd_congestion_fee	0

The January 2025 dataset was accepted because the contract explicitly permits additional columns.

3.4 Automated Testing

Automated tests were implemented using Pytest to verify the contract enforcement logic. The tests covered a missing required column, an incorrect data type, and an additional permitted column.

All three contract enforcement tests passed: test_missing_required_column_is_rejected, test_wrong_data_type_is_rejected, and test_additional_column_is_allowed. Earlier schema validation tests also passed, confirming that both breaking and compatible schema changes can be distinguished programmatically.

The tests use temporary datasets rather than modifying the original NYC TLC files.

4. Machine Learning Experiment

4.1 Model

To demonstrate the downstream effect of data-quality problems, a regression model was trained using a sample of the January 2024 NYC TLC dataset. The model used trip_distance and passenger_count as input features and fare_amount as the prediction target.

After preprocessing, 48,302 usable records were available. The data was divided into training and testing sets using an 80/20 split with a fixed random seed. A HistGradientBoostingRegressor from scikit-learn was used as the regression model. Model performance was evaluated using Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE).

4.2 Baseline Results

The baseline model was trained and evaluated using the original test features.

Metric	Baseline
MAE	2.9068
RMSE	7.3373

These values provide the reference point for the controlled data-quality experiment.

4.3 Simulated Data-Quality Failure

To investigate the potential effect of a broken data pipeline, a controlled experiment was performed in which the trip_distance feature in the test data was replaced with zero. This represents a simplified simulation of a downstream pipeline receiving an unavailable or corrupted required feature and silently substituting a default value.

Metric	Baseline	Simulated Data Problem	Change
MAE	2.9068	14.1528	+11.2460
RMSE	7.3373	20.1635	+12.8262

The MAE increased by approximately 387%, while RMSE increased by approximately 175% relative to the baseline values. The results demonstrate that corrupted input features can substantially degrade downstream model predictions.

However, this experiment is intentionally a controlled simulation. It does not demonstrate that the actual cbd_congestion_fee field introduced in January 2025 caused model degradation.

5. Contract-Enforced ML Pipeline

The final implementation integrates contract validation into the machine learning pipeline. The pipeline validates the dataset before feature preparation and model training. If required fields are missing or incompatible data types are detected, the pipeline stops rather than allowing invalid data to proceed to the model.

The contract-enforced pipeline successfully validated the January 2024 dataset before model training.

Property	Result
Contract status	VALID
Records used	48,302
Features	trip_distance, passenger_count
Target	fare_amount
MAE	2.9068
RMSE	7.3373

The pipeline therefore demonstrates a preventive approach: data validation occurs before the model consumes the data.

6. Results and Discussion

The project demonstrates that schema evolution does not inherently mean that a dataset has become incompatible.

The January 2025 NYC TLC dataset introduced cbd_congestion_fee, increasing the number of columns from 19 to 20. Since no required fields were removed and no shared data types changed, the project's data contract classified this as an acceptable schema evolution.

The automated tests further demonstrated that more serious changes can be rejected. A missing required column caused validation to fail, as did an incompatible data type. An additional permitted column was accepted.

The machine learning experiment demonstrated why these protections are useful. When the required trip_distance feature was corrupted through a controlled simulation, model error increased substantially. The increase in MAE from 2.9068 to 14.1528 shows that downstream ML systems can be highly sensitive to the quality and availability of important input features.

Therefore, schema validation should occur before data reaches downstream ML components rather than relying solely on model-level error detection.

7. Limitations

Several limitations should be considered. First, the schema comparison uses only January 2024 and January 2025 datasets. Additional monthly or yearly datasets could reveal other forms of schema evolution.

Second, the data contract is based on the January 2024 schema. A production system would require a more carefully governed process for contract versioning and approval.

Third, the machine learning experiment uses only two input features and a limited sample of the available data. The purpose of the model is to demonstrate downstream sensitivity to data-quality problems rather than to develop a production-quality fare prediction system.

Fourth, the simulated schema/data-quality failure is not the same as the actual addition of cbd_congestion_fee. The experiment demonstrates the potential consequences of corrupted required features but does not claim that the real-world addition of the congestion-fee field caused the observed degradation.

Finally, the current contract policy allows all additional columns. A production implementation could use more detailed rules to distinguish optional, deprecated, and strictly prohibited fields.

8. Responsible AI Considerations

Responsible AI principles are important when developing and evaluating machine learning systems, even when the model is used primarily for demonstration. This project emphasizes data quality, transparency, and reliability by validating input data before it reaches the ML model and by clearly documenting the limitations of the experiment. The model should not be treated as a production fare-pricing system or used to make decisions without appropriate validation and human oversight. Future deployments should

additionally evaluate potential biases in the data, monitor model performance over time, protect sensitive information, and ensure that predictions are used only for their intended purpose.

9. Conclusion

This project presents a practical approach for managing schema evolution in data pipelines using NYC TLC Yellow Taxi Trip Records.

The comparison of January 2024 and January 2025 identified a real schema change: the introduction of cbd_congestion_fee. Rather than treating this addition as an automatic failure, the implemented data contract evaluates whether required fields and data types remain compatible.

Automated tests demonstrated that breaking changes such as missing required fields and incompatible data types can be detected before reaching downstream systems. The contract-enforced ML pipeline further integrates these checks into the model workflow.

The machine learning experiment showed that a controlled corruption of an important input feature can significantly increase prediction error. This supports the need for explicit validation before data reaches machine learning systems.

Overall, the project demonstrates how schema comparison, data contracts, automated validation, and ML evaluation can work together to improve the reliability of evolving data pipelines. Future work could include contract versioning, validation across multiple years of TLC data, automated CI/CD enforcement, richer ML models, and more sophisticated compatibility rules for optional and deprecated fields.